



第14章：多模态大模型（M-LLM）架构



学习目标与主线

学习目标与主线

01

掌握从“文本大模型”向“多模态感知”的范式跨越，理解 M-LLM 如何通过核心的**模态对齐**（Alignment）技术，赋予 AI 像人类一样“看”与“听”的感官能力

02

深入剖析“视觉编码器 + 适配器 + LLM基座”的三段式**通用架构**，并透彻理解从海量图文预训练到指令微调的数据流转与参数优化过程

03

探索多模态在文档理解、智能助手及具身智能体中的**前沿应用**



引言与全景

什么是多模态大模型 (M-LLM)



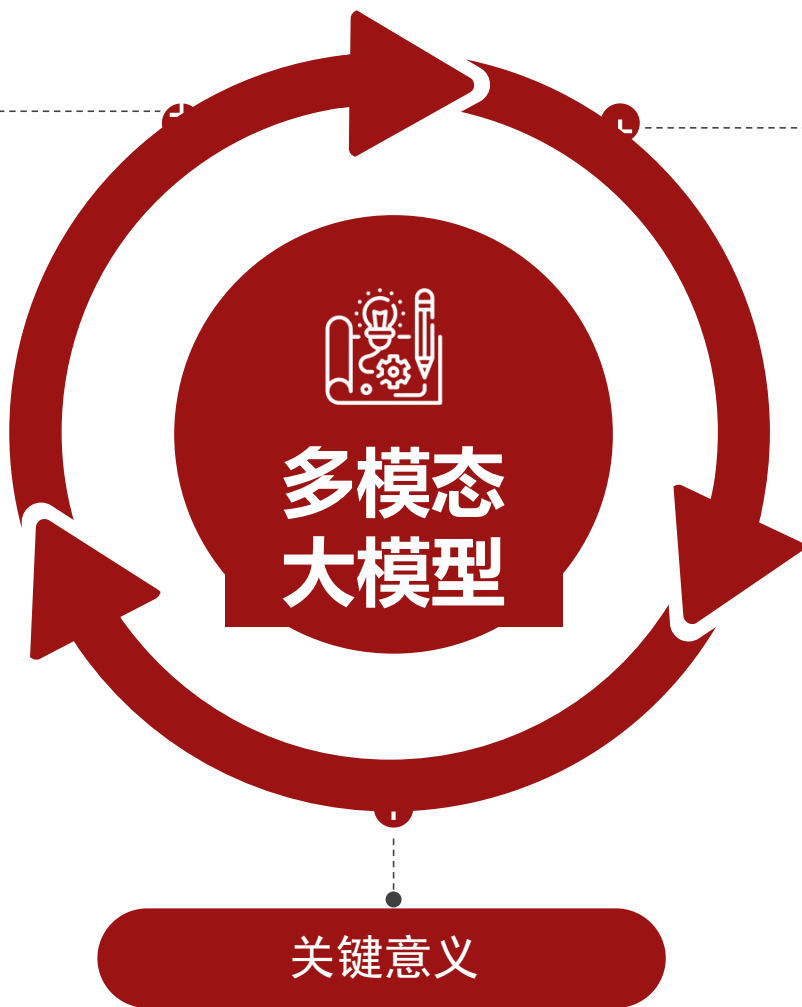
LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

核心定义

- **认知内核**: 以大语言模型 (LLM) 为“大脑”，利用其强大的泛化与推理能力。
- **感官延伸**: 通过“模态对齐 (Modality Alignment)”技术，将视觉 (Vision)、听觉 (Audio) 等连续信号映射为 LLM 可理解的“离散符号 (Tokens)”。
- **能力跃迁**: 从传统的“分类任务” (这是一只猫) 升级为“推理任务” (这只猫在干什么，为什么它看起来很生气)。



数学表述

混合模态输入 X 的条件下，预测目标序列 Y 的概率分布，其中输入 X 不再纯粹是文本，而是 $\{\text{Image}, \text{Audio}, \text{Text}\}$ 的**混合序列**

$$P(Y|X) = \prod_{i=1}^L P(y_i|X, y < i)$$

关键意义

- **通往 AGI 的必经之路**: 人类获取信息 80% 来自视觉。只有文本的模型是“缸中之脑”，M-LLM 让 AI 具备了观察物理世界的能力
 - **交互范式革命**: 从 Text-to-Text 进化为 Any-to-Any (任意模态输入，任意模态输出)

M-LLM 技术演进路线图



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

CLIP 时代

- **核心突破**：跨模态对比学习 (Contrastive Learning)
- **深度解析**：通过海量“图像-文本”对训练，将图片和文字映射到同一个高维特征空间
- **局限性**：它建立的是“相关性”。它能告诉你这张图里有一只猫，但无法像人类一样描述这只猫在做什么
- **关键词**：双塔结构、图文对齐、零样本分类

Perception • CLIP



Discriminative/判别式

BLIP-2 & LLaVA 时代

- **核心突破**：视觉特征“翻译” (Visual-Language Mapping)
- **深度解析**：利用预训练好的 LLM 作为“大脑”，通过 **Q-Former** (BLIP-2) 或 **线性投影层** (LLaVA) 作为“翻译官 (Connector)”，将复杂的视觉特征转换为 LLM 能理解的“视觉 Token”
- **里程碑意义**：LLM 第一次具备了“看图说话”和“基于视觉内容推理”的能力

Understanding • LLaVA/BLIP-2



Bridging/桥接式

GPT-4o & Gemini 1.5 Pro 时代

- **核心突破**：端到端全模态融合 (End-to-End Multimodality)
- **深度解析**：不再是拼凑不同的模型，而是在训练初期就将图片、音频、文字视为同等的信号。这解决了“翻译过程”中的信息损耗
- **体验升级**：支持任意模态输入/输出 (Any-to-Any)，响应延迟大幅降低，能够捕捉细微的情感变化和复杂的长视频逻辑
- **关键词**：统一架构、Token 化万物、极低延迟、原生跨模态推理。

Fusion • GPT-4o/Gemini



Native/原生式



核心架构设计

M-LLM 通用架构范式

三大核心组件

Modality Encoder
感官器官 (如 ViT)

Input Projector
神经束/适配器

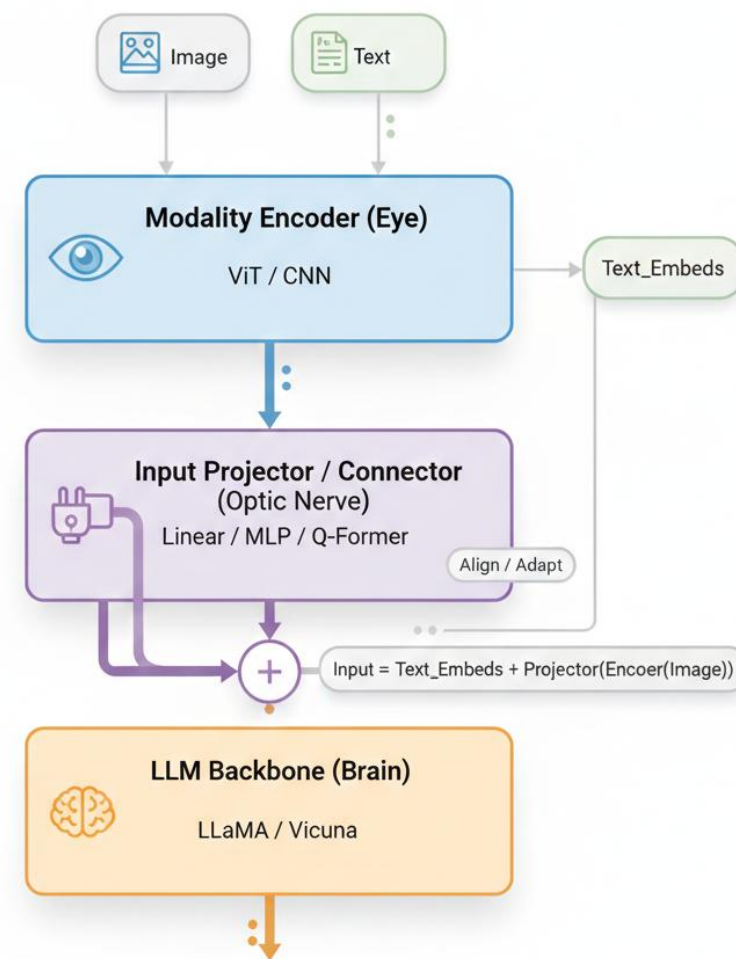
LLM Backbone
大脑 (如 LLaMA, Vicuna)

数据流向

--Image
-- Encoder
--Projector

LLM

Text Tokens



视觉编码器 (Vision Encoder) 的选择

主流架构

CLIP-ViT 是连接视觉与语言的“桥梁”。选用 L/14 或 H/14 是因为它们在模型大小和性能之间取得了最佳平衡，是目前开源界（如 LLaVA 系列）的默认配置

核心差异

为什么不用 ResNet?

ResNet (有监督)缺乏语义丰富度, CLIP (对比学习)基于海量图文对 (Image-Text Pairs) 训练, 特征空间天然与文本对齐

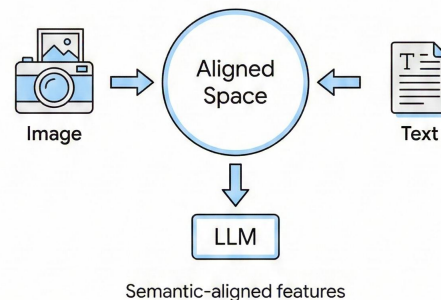
分辨率的权衡

- 逻辑链条: 高分辨率 —— 图像 Patch 数量激增 —— Visual Token 序列变长
- 显存瓶颈+训练成本

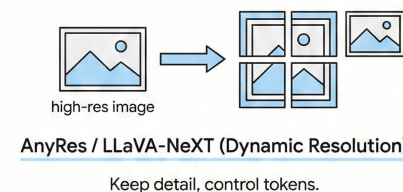
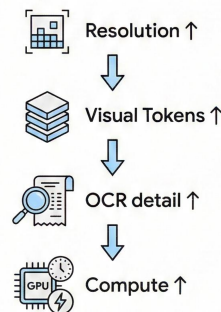
演进方向

- 传统做法: 强制导致长宽比失真或细节模糊。
- AnyRes (LLaVA-NeXT 等): 动态网格技术, 支持任意分辨率和长宽比, 大幅提升 OCR 与细节理解能力

CLIP-ViT → Aligned Space → LLM



Resolution trade-off



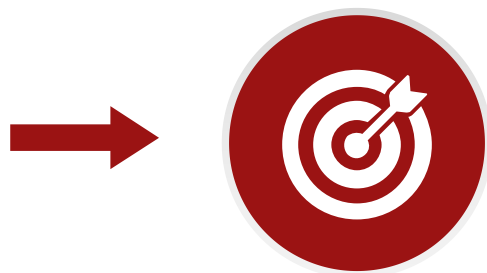
对齐方案 A —— 线性映射 (Linear Projection)

代表
模型

➤ 代表模型：
LLaVA 系列

➤ 原理：

- 1) 输入形态：视觉编码器（常见是 CLIP-ViT）把图像切成 patch，输出一串向量
- 2) 投影 (Projector)：用一个线性层或两层 MLP 把每个 v_i 映射到 LLM 的维度 d_i
- 3) 拼接进 LLM：把视觉向量当作一段前缀 token，与文本 token 拼在一起



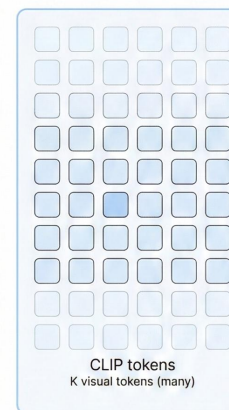
优缺点：

- 结构最简单，保留视觉信息最完整
- 但产生的 Token 数量多，推理速度慢

为什么 LLaVA 系列常用这个方案
(机制直觉)

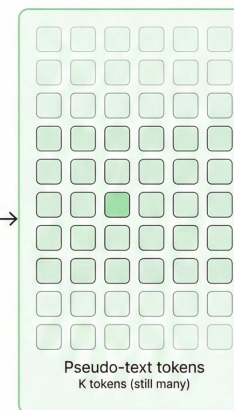
- 对齐学习主要落在 projector 上：projector 学会把“视觉 patch 表示”搬到 LLM 的语义坐标系里，使得后续语言建模损失能直接驱动“看图说话”
- 工程上通常是“两阶段”：1) 对齐预训练：图文对 (caption/对话) 让 projector 学会基础对应关系；2) 指令微调：多轮对话数据让模型学会“按指令用图回答”

Image Feature Space
(CLIP-ViT)



MLP / Linear
Projection

Text / LLM
Hidden Space



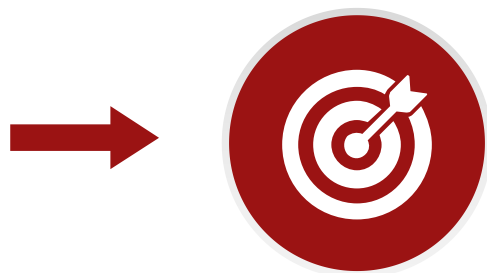
对齐方案 B —— 查询提取 (Q-Former)

代表
模型

➤ 代表模型：
BLIP-2, InstructBLIP

➤ 原理：

- 1) 输入：密集视觉 tokens，视觉编码器先产生大量 image tokens
- 2) 关键组件：可学习 Queries (Q)，模型参数本身
- 3) Cross-Attention：用 Q 去读 V，每个 query 关注图像中不同区域/语义（物体、关系、属性），把信息汇总到 query 的输出里
- 4) 再映射到 LLM，把 Q 过一层线性/MLP 变到 LLM hidden size，然后作为视觉前缀输入 LLM

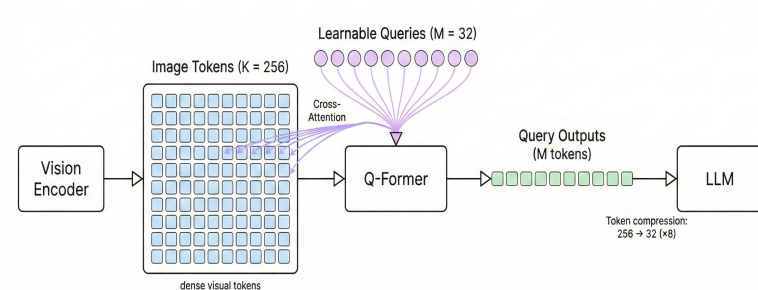


优缺点：

- Token 压缩率极高（如 256——32），过滤背景噪声，推理加快
- 但可能丢失细粒度信息

适用场景

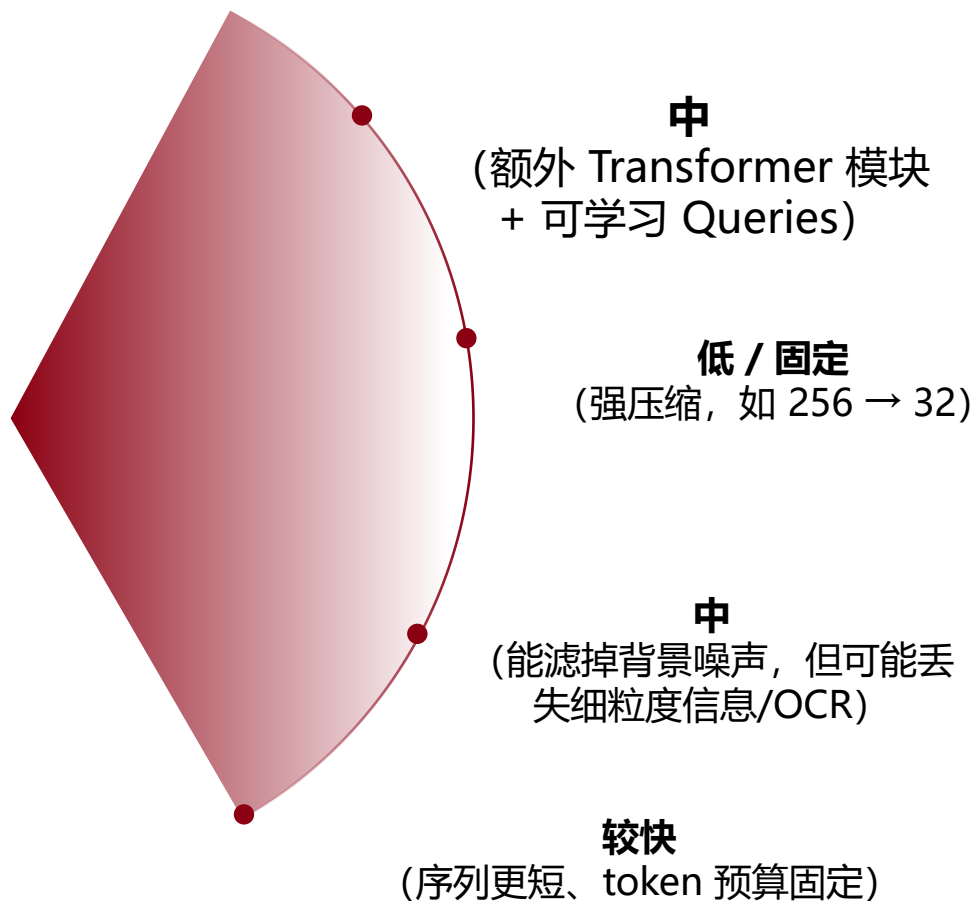
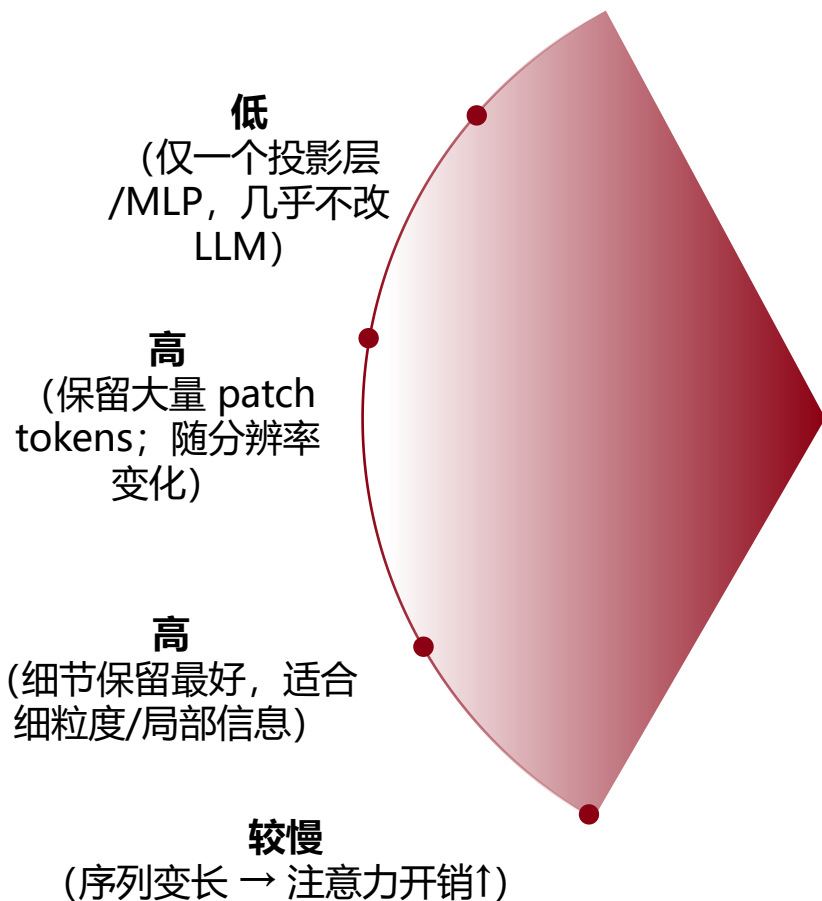
- 更适合：VQA、图文对话、通用语义理解（需要“抓重点”的任务）
- 相对吃亏：OCR、细粒度定位、需要完整像素细节的任务



架构对比总结

MLP / 线性映射

Q-Former / 查询提取



结构复杂度

输入到 LLM 的视觉 Token 数量

信息完整度 (保真)

推理速度



训练范式与实战

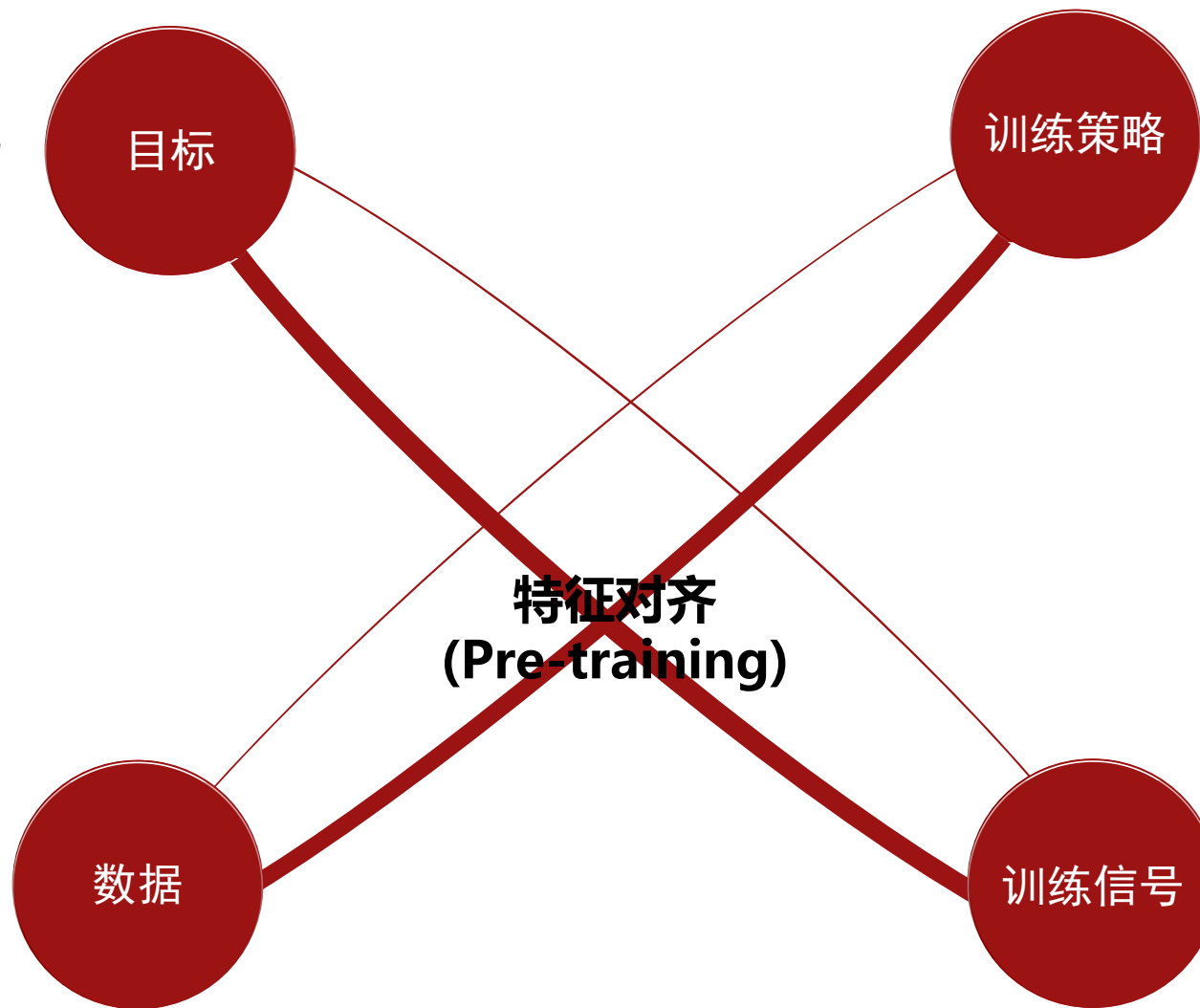
训练阶段 I —— 特征对齐 (Pre-training)

这一步到底在“对齐”什么？

- **核心矛盾**：视觉编码器（如 CLIP-ViT）输出的是一串 patch-level 视觉特征（continuous embeddings），而 LLM 期望输入的是 token embedding 序列（位于 LLM 的语义/词向量空间）
- **Feature Alignment 的本质**：把视觉表示翻译到 LLM 的输入分布里，让 LLM 开始“看得懂图”

为什么用海量图文对 (Image-Text Pairs) ？

- 图文对（网页 alt-text、标题、描述等）提供了**最直接的监督信号**——图像内容 ↔ 对应文本描述
- 这类数据规模可以**做到极大**（如 LAION-5B 一类 Web-scale 数据），非常适合做“让模型先学会把图像映射到可语言化语义”的基础对齐



为什么冻结 Encoder 和 LLM，只训练 Connector？

- **目的**：把训练的“自由度”压到最小，只解决一个问题：把视觉特征对齐到 LLM 可用的输入空间
- **三点收益**：1) 稳定：冻结大模型避免语言能力漂移/灾难性遗忘；2) 高效：只训练小模块，显存与算力成本显著下降；3) 解耦：Encoder 负责“看清楚”，LLM 负责“会说话”，Connector 负责“翻译协议”

怎么让 Connector 学会对齐？

- 最常见做法是直接利用语言模型的 **next-token 目标** (LM loss)：
 - 输入：[视觉tokens] + (caption 文本 tokens)
 - 目标：让 LLM 生成/预测 caption（或补全文本）
- 直觉上：如果 Connector 把图像信息“翻译得好”，LLM 就更容易预测正确的描述文本；梯度就会推动 Connector 学到更好的映射

训练阶段 II —— 指令微调 (Instruction Tuning)



Stage II 把模型从“会看会说”升级为“会按人类要求办事”

目标

Instruction Following

- 指令遵循不是“更懂图”，而是更懂人类意图
- 对话：问答、追问、纠错、多轮一致性
- 推理：比较、计数、空间关系、因果解释、一步步推导

Stage I (特征对齐) 让模型做到“看图→能描述”，但它学到的是图文共现统计，更像“自动写 caption”

数据

为什么强调“高质量多模态对话”？

- 典型数据形态从“image-caption”变成“image + instruction → response”：
- VQA：图像 + 问题 → 答案（适合训练“问答对齐”）
- Visual Dialogue：多轮问答（适合训练“多轮上下文”）

数据质量会直接塑造模型的行为模式：是否啰嗦、是否乱猜、是否遵循格式、是否能承认不确定

关键点

为什么必须配合特定的对话格式 (Prompt Format)

- 指令微调时，数据通常被包装成固定模板，例如：
 - System: 规则/身份
 - User: 问题（含图像占位符）
 - Assistant: 标准回答

模型会把这种模板当成“启动协议”：哪里开始回答、用什么语气、如何引用图片、如何分段，都被写进了训练分布

总结

指令微调会固化模型的对话‘协议’；推理时必须使用匹配的 Prompt Format，否则容易出现行为漂移与能力下降

指令遵循的桥梁：系统指令与对话格式

为什么说它是“桥梁”？

- 在多模态指令微调 (Slide 9) 之后，模型不只是学“图像→文本”，更学会了一套对话协议
- 系统指令 + 对话格式就是把“人类意图”以模型最熟悉的方式编码进输入序列，从而触发正确的行为模式（按指令、按格式、按角色）

不同的提示结构

Chat Template

Chat Template 定义了序列结构

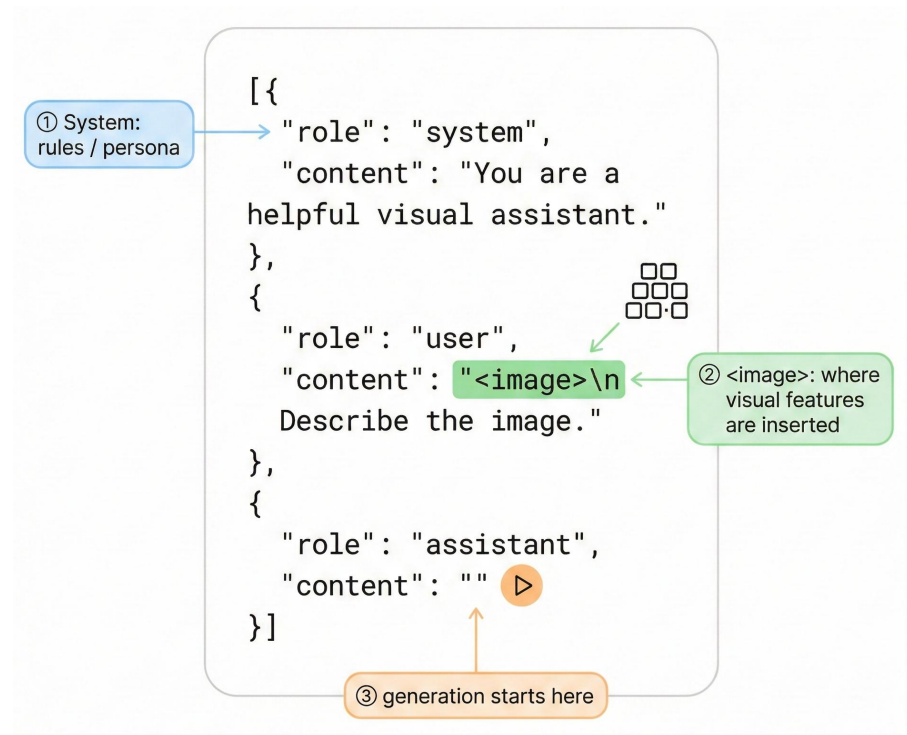
- <system> ... <user>
<image_placeholder> ...
<assistant> ...

System Prompt

- System Prompt 是最高优先级的行为约束（相当于“总规则/身份设定”），典型作用包括
- 角色与语气：视觉助手、简洁/详细、是否解释推理
 - 安全与边界：不能做什么、遇到不确定要怎么讲
 - 输出规范：必须给 JSON、必须分点、必须引用证据

特殊 Token

<image> / <image_placeholder>
它告诉模型：在序列的这个位置插入视觉特征（visual embeddings）





能力扩展与评估

视觉理解的高级能力

把“图像”变成可推理的“文本证据”

- **需求：**传统“看图”主要捕捉物体、场景、关系；但真实世界里大量关键信息在文字里——文档/票据/合同扫描件等
- **OCR 类模型能力要求：**识别文本内容（读对字）；保持结构（表格行列、段落层级、图表标签）；与指令对齐（按要求抽取字段、对比数值、生成结构化输出）
- **为什么难：**文字通常小且密，需要更高分辨率/更多 visual tokens；易受字体、模糊、扭曲、遮挡影响；还要跨模态对齐



Text:
Total: 128

OCR（文字识别）

把“语言指令”落到“图像坐标”

- **解决什么：**从回答“是什么”升级到回答“在哪里”
- **例子：**“把左上角红色按钮框出来”、“指出图中异常区域的位置”
- **输出常见形式：**Bounding box-[x1, y1, x2, y2]（通常是像素或归一化坐标）
- **核心机制：**模型需要把语言中的指代（left/top/第二个/靠近...）映射到视觉空间——视觉侧要保留足够的空间分辨率、语言侧要学会“引用空间位置”并输出可计算的坐标



[x1, y1,
x2, y2]

Grounding（空间定位）

理解时间序列和因果关系

- **解决什么：**
 - **事件跟踪：**同一对象跨帧的状态变化（移动、变形、出现/消失）
 - **因果与顺序：**先发生什么、导致了什么（Before/After）
 - **长上下文对齐：**把多帧信息压缩成可用记忆，不被 token 爆炸拖垮
- **为什么难：**主要瓶颈是计算与上下文长度：帧数 × 分辨率 × tokens 会指数式变大；需要“选帧/压缩/汇聚”



多图/视频流

音频与视频大模型 (Audio & Video)

Video-LLM

把“帧序列”变成可推理的时间表征

➤ 视频相比图片多了什么？

- 1) 运动与变化 (动作、轨迹、状态变化)
- 2) 事件边界 (什么时候开始/结束)
- 3) 因果/顺序 (先发生什么导致什么)

➤ 为什么需要 Time-Adapter (时间适配器)？

- 1) 直接把每帧都当图片喂进去会导致 token 爆炸 (帧数 × 每帧视觉 tokens)，**计算不可控**
- 2) Time-Adapter 的作用：在时间维度做“**压缩与对齐**”，让模型既保留关键动态信息，又能在 LLM 的上下文长度内工作

Speech-LLM

级联 vs 原生

➤ 级联模式 (Cascaded)：ASR → LLM → TTS

- 1) ASR (语音转文字)
- 2) TTS (文字转语音)
- 3) 关键缺陷：信息丢失 (语音 ≠ 文字) —— 天然会丢掉大量副语言信息：语气、情绪、强弱、讽刺、停顿、重音、语速变化

➤ 原生模式 (Native)：Audio Tokens 直接入模 (如 GPT-4o)

- 1) 不把音频先“翻译成纯文本”，而是把音频编码成 audio tokens (离散或连续表示)，直接和文本 token 一起在同一模型里做理解与生成
- 2) 优势：全信息保留、低延迟与更顺滑的交互
- 3) 代价：训练与推理更复杂、算力更大；对数据质量和对齐训练 (音频-文本-行为) 要求更高



多模态从“图像”走向“时间信号”：视频=时序视觉，语音=连续声学交互范式革命

图文生成 (Visual Generation)

路线 1：工具调用

1

原理：LLM 当“导演”，扩散模型当“画师”

用户意图 → LLM 写出精确的 Text Prompt (+参数) → 调用 Stable Diffusion / DALL-E 等图像生成器 → 返回图片

2

优势

- 1) 质量强、生态成熟：扩散模型专精图像质量、细节和风格控制
- 2) 工程解耦：LLM 不需要学会“像素生成”，只要学会写 prompt + 调参数

3

局限

- 1) 一致性与可控链路更长：多轮编辑/保持角色一致，需要额外机制（参考图、ControlNet、IP-Adapter、LoRA 等）。
- 2) 交互延迟：生成往往比纯文本慢，且系统是“多组件拼装”

路线 2：原生生成

1

原理：把图像离散，再让 LLM 直接预测

先用 VQ-VAE / tokenizer 把图像压缩编码为离散 code，LLM 像生成文本一样生成这些 image tokens，然后再解码还原

2

优势

- 1) 统一序列、天然多模态混合，更适合多轮编辑以及图文混合输出（边讲边画）
- 2) 更像“端到端”：不需要外部图像引擎的接口协调

3

局限

- 1) 训练难、成本高：要同时学语言与图像生成，数据与算力要求大
- 2) 图像质量/分辨率可能受限

总结

LLM 只会预测 token；“画图”要么调用外部图像模型，要么把图像 token 化后让 LLM 直接生成

【评估】模型基准与生态 (Benchmarks)

学术
基准

- MMMU: 专家级多学科知识 (最硬核)
- MathVista: 视觉数学推
- POPE: 幻觉 (Object Hallucination) 测试



基准
指标

生态
指标

- 可用性:
HuggingFace/Transformers 接入、Web Demo、推理脚本是否成熟
- 格式稳定性:
是否强依赖特定 chat template / 特殊 token (不一致就掉点)
- 可微调+部署成本

Benchmark (指标)	Metric	GPT-4o	LLaVA-NeXT (≈LLaVA-1.6-34B)	Qwen-VL (家族代表)
MMMU (Val 900)	Acc ↑	69.1 (Hugging Face)	51.1 (Hugging Face)	51.4 (Qwen-VL-MAX) (Hugging Face)
MathVista (Test)	Acc ↑	63.8 (GitHub)	46.5 (GitHub)	43.3 (Qwen-VL-Plus) (GitHub)
POPE (幻觉)	F1 ↑	85.0 (arXiv)	87.73 (LLaVA)	82.3 (arXiv)

[1] <https://huggingface.co/datasets/MMMU/MMMU>

[2] <https://github.com/lupantech/MathVista>

[3] <https://arxiv.org/html/2410.11701v1>

[4] <https://llava-vl.github.io/blog/2024-01-30-llava-next/>



挑战、前沿与总结

典型应用场景全景



典型应用场景全景

多模态大模型把“看、听、读、写”合成一个**统一能力栈**：理解多源输入 → 结构化抽取 → 推理决策 → 生成输出/执行动作，从而覆盖个人、企业、创意三类高频落地场景

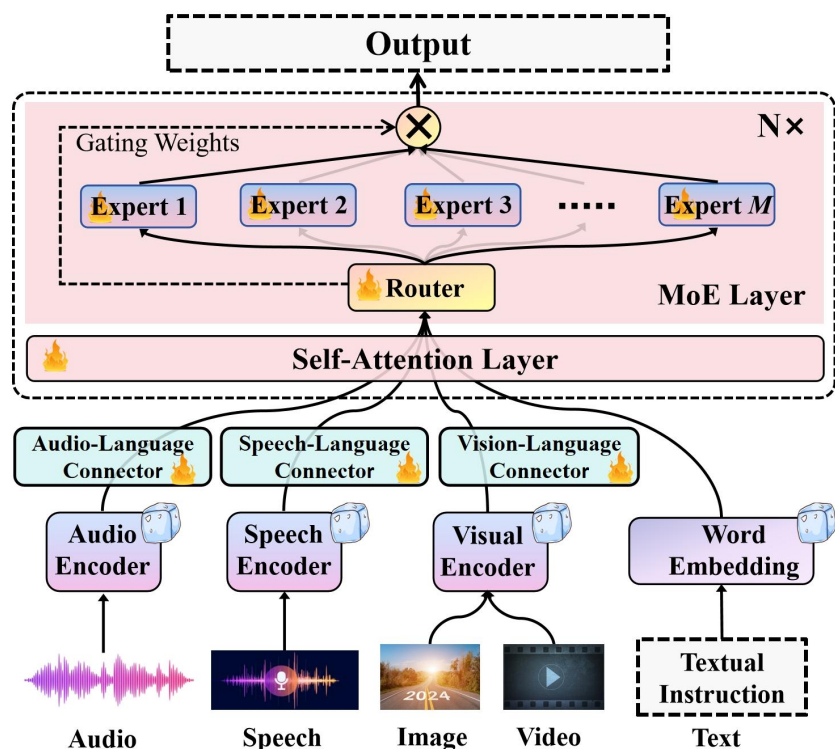
✓ 视障辅助（Be My Eyes 类） 视障辅助中，用户用手机拍照/实时画面输入，模型完成视觉理解与关键信息抽取，必要时主动追问澄清，并输出可执行的行动建议，实现“看见→解释→追问→指导”的闭环 ✓ 智能识屏助手（手机/桌面） 智能识屏助手从截图/屏幕流中理解UI元素与布局，匹配用户意图并给出分步操作；关键是强OCR+版式解析与低幻觉，仅基于可见内容回答	✓ RAG 文档理解（发票、合同、财报表格） 通常以 PDF/扫描件/拍照 + 企业知识库为输入，按 解析 (OCR+版式) → 字段结构化抽取 → RAG检索制度口径 → 规则校验与计算 → 输出可审计结果（字段-证据-规则）的流水线落地 ✓ 工业质检（视觉检测 + 语义解释） 工业质检用产线图像/视频做缺陷定位与分类，并输出可解释的处置建议	✓ Sketch-to-Code（草图变代码 / UI 生成） 以手绘草图/线框图加文字约束为输入，生成可直接使用并可迭代修改的前端组件代码（含交互） ✓ 视频自动剪辑（多模态检索 + 叙事生成） 视频自动剪辑以长视频配合目标与风格约束为输入，模型先理解镜头语义并检索亮点片段，再完成时间线编排与文案/字幕生成，输出剪辑方案。落地通常先产出可审稿的分镜脚本，再一键渲染成片以降低创作风险
---	---	---

个人场景：无障碍与日常感知增强	企业场景：RAG 文档理解 + 生产流程自动化	创意场景：从自然输入到可编辑产物
-----------------	-------------------------	------------------

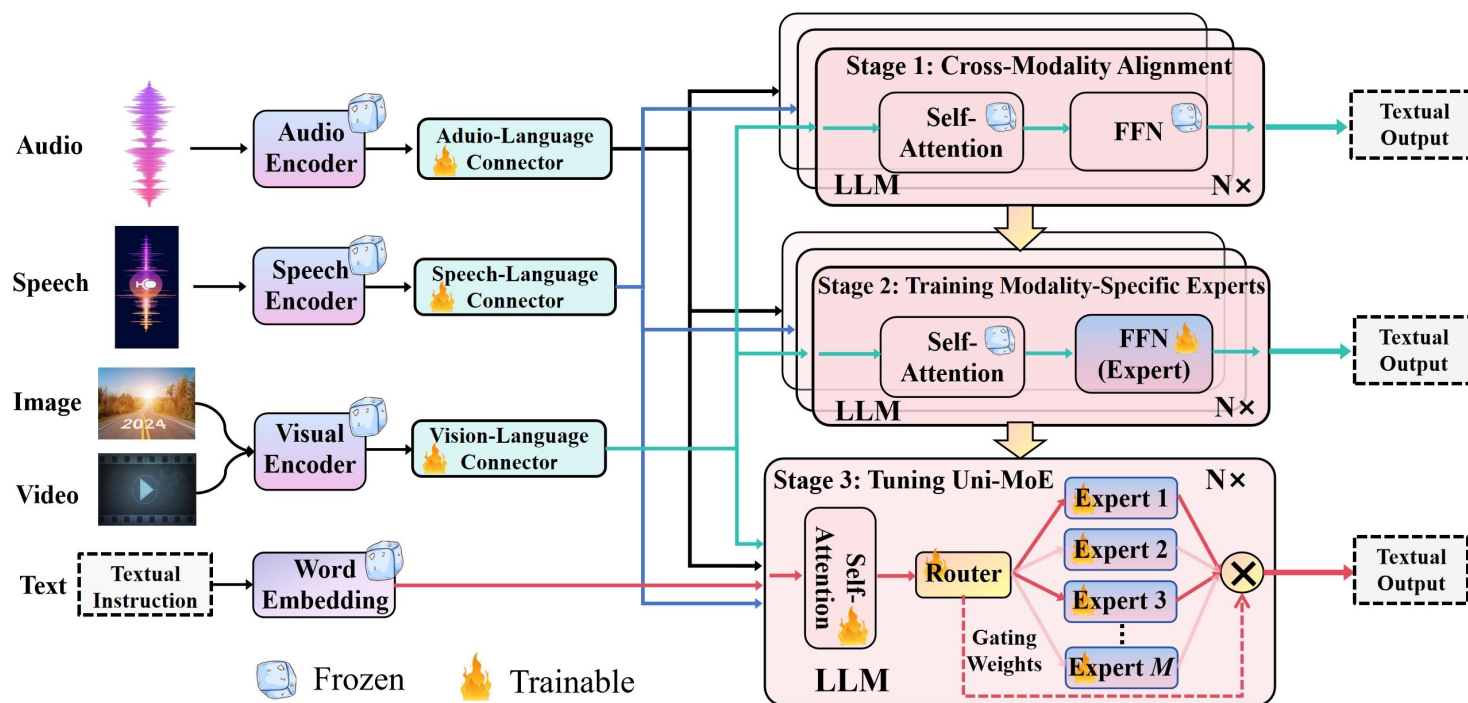
多模态应用落地的共同模式是：感知输入（图/文/音/视频）→ 结构化与检索增强（RAG）→ 可靠推理与可审计输出 → 对话式迭代

经典开源案例—— Uni-MoE

以大语言模型为核心，提出了多模态数据理解的多专家混合架构，构建了统一多模态大模型Uni-MoE



模型架构



渐进式三阶段训练策略

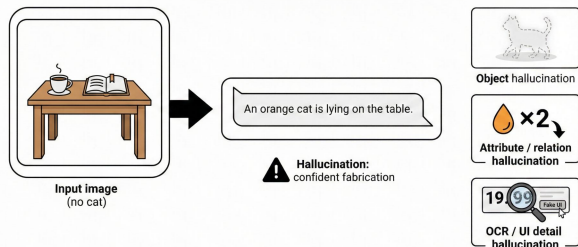
核心挑战 —— 幻觉 (Hallucination)

本质

模型在视觉证据不足或对齐不稳时，仍沿着“最像人话”的自回归惯性继续编造，从而输出与图像不一致的细节

现象

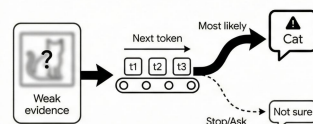
典型表现：图里没有猫，却“非常肯定”地描述猫的颜色、动作、位置



成因

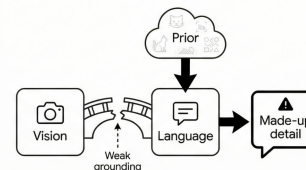
- 自回归惯性
- 视觉-语言对齐不充分 + 数据偏置

A) Autoregressive momentum



自回归惯性

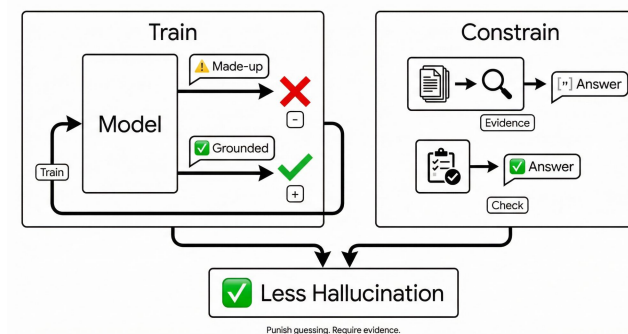
B) Weak grounding + prior bias



数据偏置

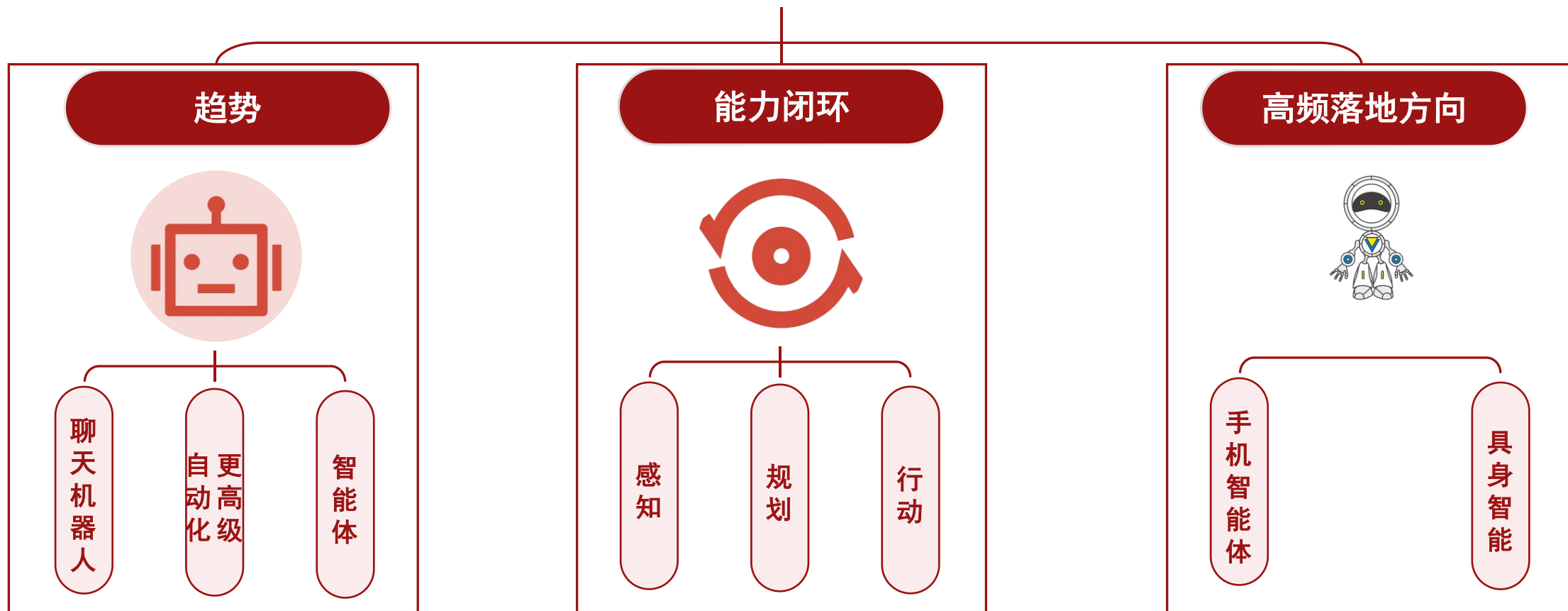
怎么解决

- 训练端：让模型“更愿意看证据、更敢说不知道”
- 推理端 (Inference-Time)：不改模型或少改模型，靠“约束与验证”降幻觉



未来展望 —— 自主智能体

下一阶段的 M-LLM 不再只“回答”，而是把模型放进一个**感知→规划→行动**的闭环里，能在真实环境（手机/机器人）中**自主分解任务、调用工具、执行并纠错**



不仅说“怎么做”，还会去做、做完
回报、失败再试

实际系统里通常还会加
Memory/Feedback (短期工作记忆+
长期经验/规则)，让 agent 越做越稳

像“真人用户”一样操作 App
+
把“语言规划”落到机器人动作

01

架构主线：ViT 负责感知，Connector 做对齐，LLM 负责推理与生成

02

效果关键：对齐与高质量数据决定上限，让模型“看证据再说”

03

发展方向：接入工具/智能体闭环，把认知内核连到物理世界